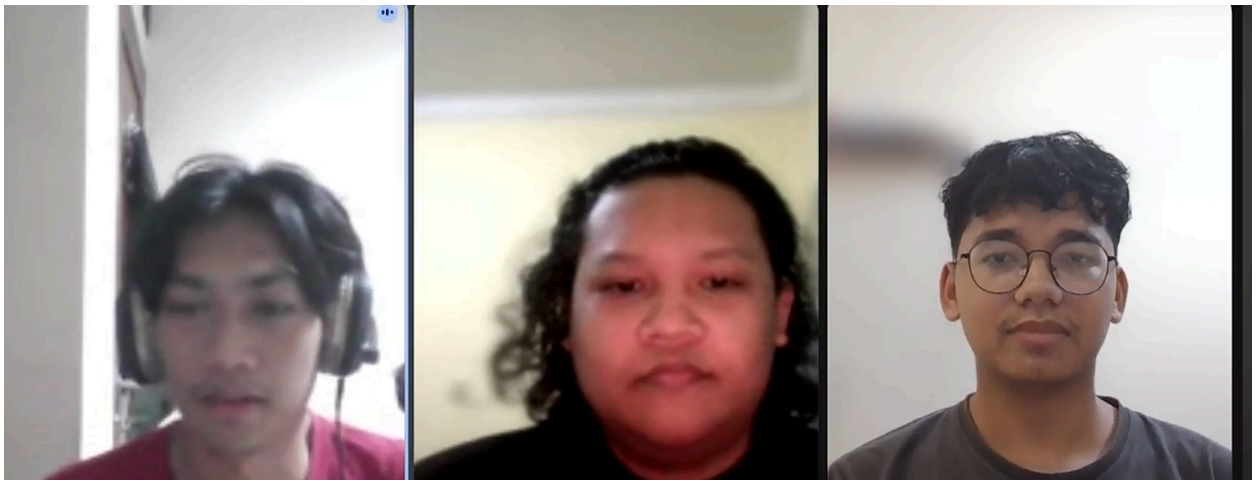
Laporan Tugas Besar II
IF 2211-Strategi Algoritma

Disusun oleh:

sbrep2



Anggota:

1. **13524022 Ishak Palentino Napitupulu**
2. **13524112 Richard Samuel Simanullang**
3. **13524118 Dylan Gregory Tondang**

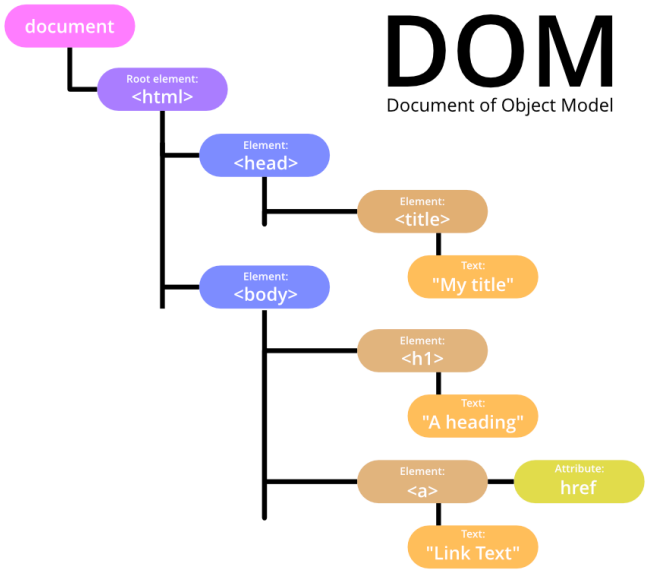
Teknik Informatika
Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Bab I: Deskripsi Tugas

Deskripsi Tugas



Gambar 1. Struktur Pohon DOM

https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

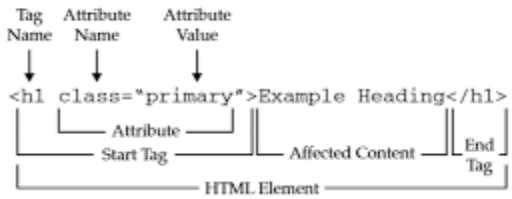
Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. **Tags**



Gambar 2. Struktur Syntax HTML

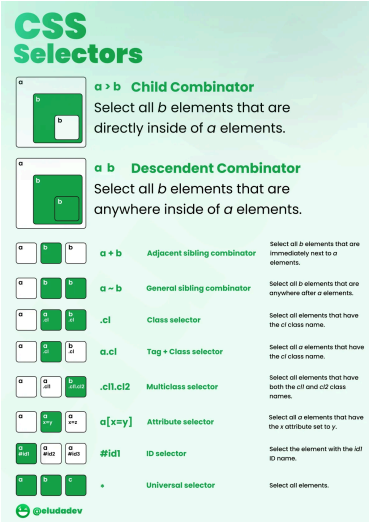
https://tutorial.techaltum.com/htmlTags.html

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang *self-closing*.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling . Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. **CSS Selector**



Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector

  Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen <p>.
- Class Selector

  Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.).Contoh: .box
- ID Selector

  Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#).

  Contoh: #header
- Universal Selector

  Memilih semua elemen HTML.

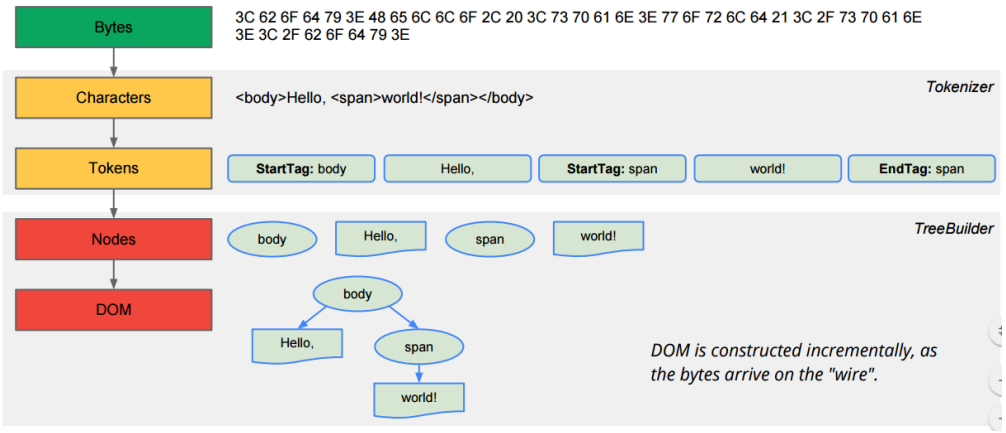
  Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (“ “)

- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)

3. HTML Parsing



Gambar 4. Proses Parsing Pohon HTML

https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen <html>. Dari node root tersebut, elemen-elemen lain seperti <head> dan <body> akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

Bab II: Landasan Teori

Graf

Graf merupakan struktur data yang terdiri dari kumpulan simpul (node) dan sisi (edge) yang menghubungkan antar simpul. Setiap node merepresentasikan suatu nilai, kondisi, atau entitas tertentu, sedangkan edge berfungsi sebagai penghubung antar node. Dalam graf, dikenal pula istilah path, yaitu jalur yang menghubungkan satu node ke node lainnya, di mana path yang berawal dan berakhir pada node yang sama disebut sebagai siklus. Berdasarkan arah pada setiap sisinya, graf dibedakan menjadi graf berarah (directed graph), yaitu graf yang setiap sisinya memiliki arah tertentu, dan graf tak berarah (undirected graph), yaitu graf yang tidak memiliki arah pada sisinya.

Tree

Tree merupakan salah satu bentuk khusus dari graf yang bersifat non-linier dan memiliki struktur hierarki. Tree terdiri dari sekumpulan node yang saling terhubung tanpa membentuk siklus. Node paling atas dalam tree disebut root, yang menjadi titik awal dari struktur tersebut. Setiap node dapat memiliki anak (child), dan jumlah anak dari suatu node disebut degree. Selain itu, dikenal pula istilah depth yang menyatakan jarak atau tingkat kedalaman suatu node dari root. Tree yang setiap nodenya memiliki maksimal dua anak disebut sebagai binary tree. Struktur tree banyak digunakan untuk merepresentasikan hubungan hierarkis, seperti struktur organisasi atau proses pengambilan keputusan.

Traversal Graf

Dalam konteks pencarian solusi, graf digunakan sebagai representasi permasalahan, di mana setiap node menggambarkan suatu keadaan (state) dan edge menunjukkan kemungkinan perpindahan antar keadaan. Proses traversal graf adalah teknik untuk mengunjungi node-node secara sistematis guna menemukan solusi dari suatu permasalahan. Berdasarkan ketersediaan informasi, metode pencarian pada graf dibagi menjadi dua, yaitu pencarian tanpa informasi (uninformed search) yang tidak menggunakan pengetahuan tambahan tentang solusi, serta pencarian dengan informasi (informed search) yang memanfaatkan heuristik untuk mempercepat pencarian. Selain itu, graf dalam proses pencarian dapat direpresentasikan secara statis, yaitu sudah terbentuk sebelum proses pencarian, atau secara dinamis, yaitu dibangun selama proses pencarian berlangsung. Pendekatan ini menjadi dasar dalam pengembangan algoritma pencarian seperti BFS dan DFS yang akan dibahas lebih lanjut.

DOM

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram. Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Algoritma BFS

Breadth First Search (BFS) merupakan metode pencarian yang melakukan penelusuran secara melebar, yaitu dengan mengunjungi semua simpul yang bertetangga langsung dengan simpul awal terlebih dahulu sebelum melanjutkan ke simpul pada tingkat berikutnya. Algoritma :

1. Kunjungi simpul v
2. Kunjungi semua simpul yang bertetangga dengan simpul v terlebih dahulu
3. Kunjungi simpul yang belum dikunjungi dan bertetangga dengan simpul-simpul yang tadi dikunjungi, demikian seterusnya

Dalam implementasinya, BFS umumnya menggunakan struktur data antrian (queue) untuk menyimpan simpul yang akan dikunjungi berikutnya, serta struktur tambahan untuk menandai simpul yang sudah dikunjungi agar tidak terjadi pengulangan.

Algoritma DFS

Depth First Search (DFS) merupakan metode pencarian yang melakukan penelusuran secara mendalam. Algoritma ini akan mengunjungi satu simpul dan kemudian melanjutkan ke salah satu simpul tetangganya secara terus-menerus hingga mencapai simpul yang tidak memiliki tetangga yang belum dikunjungi. Algoritma :

1. Kunjungi v
2. Kunjungi simpul w yang bertetangga dengan simpul v
3. Ulangi DFS mulai dari simpul w

4. Ketika mencapai simpul u sedemikian sehingga semua simpul yang bertetangga dengannya telah dikunjungi, pencarian dirunut-balik (backtrack) ke simpul terakhir yang dikunjungi sebelumnya dan mempunyai simpul w yang belum dikunjungi.
5. Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi.

Proses ini berlanjut hingga seluruh simpul yang dapat dicapai telah dikunjungi. DFS biasanya diimplementasikan menggunakan rekursi atau struktur data stack .

Bab III: Aplikasi BFS dan DFS

Aplikasi BFS

Algoritma BFS dalam program kami mengeksplorasi pohon DOM secara melebar (level demi level). Artinya, program akan mengecek elemen induk (root), lalu turun mengecek seluruh elemen anak di bawahnya secara berurutan, lalu cucu-cucunya, dan seterusnya. Berikut langkah demi langkah algoritma BFS :

1. Menginisialisasi queue

   Jika root tidak kosong, program menginisialisasi queue (antrean) berupa slice/array yang pertama kali berisi node root. Disiapkan juga wadah untuk visited (menyimpan urutan node yang pernah dikunjungi), matches (node yang cocok), dan log (catatan teks eksekusi).

2. Main traversal

   Fungsi akan mengambil elemen posisi paling depan di queue (yakni node := queue[0]), dan langsung menghapusnya dari queue (queue = queue[1:]). Prinsip ini adalah First In First Out (FIFO).

3. Pencatatan Log

   Node yang baru saja diambil dari antrean akan dimasukkan ke visited dan dicatat dalam riwayat log dengan teks format "Visit: [nama_node] (depth=[kedalaman])".

4. Pengecekan Selector

   Program memanggil fungsi selector.ComboCombinator(node, selectors) untuk mengecek apakah elemen tersebut sesuai dengan CSS selector yang diminta.

   - Jika cocok maka elemen saat itu ditambahkan ke dalam daftar matches dan dicatat ke dalam log sebagai "Match".
   - Jika ada input jumlah maksimum hasil pencarian (limit > 0), program akan mengecek apakah total matches sudah mencapai batas (count == limit). Jika sudah, program langsung berhenti seketika (return), sehingga menghemat waktu dan komputasi (pohon tersisa tidak ditelusuri).

5. Tambahkan child ke queue

   Jika batas limit belum tercapai, fungsi akan ambil semua simpul anak (node.Children) dari node yang sedang diproses secara berurutan dari kiri ke kanan, dan menambahkannya ke bagian tail dari queue.

6. Proses ini berulang ke tahap 2 sampai queue habis atau dalam kata lain, semua node di tree-nya sudah dicek.

Penyusunan antrean (queue) pada algoritma BFS di program ini memanfaatkan tipe data Slice bawaan dari bahasa Go ([]*parser.Node). Karena Go tidak memiliki struktur data queue, program ini mensimulasikan mekanisme antrean menggunakan operasi slicing dan append. Berikut pseudocode untuk BFS :

```
FUNCTION BFS(limit, root, selectors):
    IF root IS NULL:
        RETURN (NULL, NULL, NULL)


    queue ← list berisi root
    visited ← list kosong
    matches ← list kosong
    log ← list kosong
    count ← 0


    WHILE queue tidak kosong:
        node ← elemen pertama queue
        HAPUS elemen pertama dari queue


        TAMBAHKAN node ke visited
        TAMBAHKAN "Visit: nodeLabel(node), depth=node.Depth" ke log


        IF selector.ComboCombinator(node, selectors) = TRUE:
            TAMBAHKAN node ke matches
            count ← count + 1
            TAMBAHKAN "Match: nodeLabel(node), depth=node.Depth" ke
log


            IF limit > 0 DAN count = limit:
                RETURN (matches, visited, log)


        UNTUK tiap child dalam node.Children:
            TAMBAHKAN child ke queue


    RETURN (matches, visited, log)
```

Aplikasi DFS

Algoritma DFS mengeksplorasi pohon DOM secara mendalam. Artinya, program akan mengeksplorasi satu rantai cabang terus ke bawah sampai mencapai daun yang tidak punya anak lagi, sebelum backtrack ke atas dan eksplor cabang sebelahnya. Algoritma DFS ini menggunakan cara rekursif yang dapat dijelaskan seperti berikut. Berikut langkah demi langkah algoritma DFS yang ada di program kami :

1. Inisialisasi Struktur Penyimpanan

   Fungsi DFS pertama-tama menginisialisasi tiga buah list, yaitu:
   - matches untuk menyimpan node yang memenuhi kriteria selector,
   - visited untuk menyimpan node yang telah dikunjungi,
   - log untuk mencatat proses traversal. Setelah itu, fungsi rekursif dipanggil dengan node awal (root).
2. Pengecekan Node Kosong (Base Case)

   Pada fungsi rekursif, langkah pertama adalah cek apakah node nil. Jika ya, fungsi langsung mengembalikan nilai false sebagai tanda tidak ada proses lebih lanjut pada cabang tersebut.
3. Mengunjungi Node (Visit Node)

   Jika node tidak kosong, maka:
   a. Node ditambahkan ke dalam list visited.
   b. Informasi kunjungan node (label dan depth) dicatat ke dalam log.
4. Pengecekan Kesesuaian Selector (Matching)

   Node yang sedang dikunjungi kemudian diperiksa apakah memenuhi kriteria selector menggunakan fungsi ComboCombinator.
   a. Jika memenuhi, node ditambahkan ke dalam list matches.
   b. Informasi kecocokan tersebut juga dicatat ke dalam log.
5. Pengecekan Batas (Limit)

   Setelah ditemukan node yang cocok, dilakukan pengecekan apakah jumlah node pada matches sudah mencapai batas (limit).
   a. Jika limit > 0 dan jumlah matches sama dengan limit, maka fungsi mengembalikan true.
   b. Nilai true ini digunakan untuk menghentikan proses traversal lebih awal (early stopping).
6. Eksplorasi Anak Node (Rekursi ke Bawah)

   Jika limit belum tercapai, algoritma akan melanjutkan traversal ke setiap anak dari node saat ini:
   a. Untuk setiap child dalam node.Children, fungsi rekursif dipanggil kembali secara rekursif.

b. Jika salah satu pemanggilan rekursif mengembalikan true, maka proses traversal dihentikan dan nilai tersebut langsung dikembalikan ke pemanggil sebelumnya.

7. Backtracking

   Jika seluruh child telah dieksplorasi dan tidak ada kondisi penghentian, maka fungsi mengembalikan false.

   Proses ini merupakan bentuk *backtracking*, di mana algoritma kembali ke node sebelumnya untuk melanjutkan eksplorasi cabang lain.

8. Hasil Akhir

   Setelah seluruh proses traversal selesai atau kena prune, fungsi DFS mengembalikan:

   a. daftar node yang cocok (matches),
   b. daftar node yang dikunjungi (visited),
   c. serta log proses traversal (log).

Berikut pseudocode dari DFS

```
FUNCTION DFS(limit, root, selectors):
   matches ← list kosong
   visited ← list kosong
   log ← list kosong

   PANGGIL dfsRecv(root, selectors, limit, matches, visited, log)

   RETURN (matches, visited, log)

FUNCTION dfsRecursive(node, selectors, limit, matches, visited, log) -> boolean:

   IF node IS NULL:
      RETURN FALSE
   TAMBAHKAN node ke visited
   TAMBAHKAN "Visit: nodeLabel(node), depth=node.Depth" ke log
   IF selector.ComboCombinator(node, selectors) = TRUE:
      TAMBAHKAN node ke matches
      TAMBAHKAN "Match: nodeLabel(node), depth=node.Depth" ke log
      IF limit > 0 DAN ukuran(matches) = limit:
         RETURN TRUE

   UNTUK setiap child dalam node.Children:
      IF dfsRecursive(child, selectors, limit, matches, visited, log) = TRUE:
         RETURN TRUE

   RETURN FALSE
```

Bab IV: Implementasi dan Pengujian

Penjelasan Kode Sumber

Program ini dimulai dengan fungsi utama BackEnd menerima request dari FrontEnd, yang berisi :

1. url atau html : Tautan website yang ingin ditelusuri atau teks html raw
2. algorithm : Pilihan algoritma traversal
3. selector : teks css selector yang akan dicari
4. limit : Batasan hasil yang ingin dicari

Kemudian, jika user memasukkan url, maka program akan menggunakan fungsi Scrape di scraper untuk melakukan HTTP GET request ke URL dan mengambil keseluruhan teks HTML-nya.

Teks HTML yang didapat kemudian akan dimasukkan ke dalam modul parser (parser.Parse). Pada tahap ini, program memecah string HTML menjadi token-token (Tag Buka, Tag Tutup, dll) menggunakan html.Tokenizer. Program akan membuat struktur data pohon (DOM Tree) secara manual (berupa struktur bersarang parser.Node). Node html akan bertindak sebagai root. Pembentukan urutan elemen (siapa yang jadi parent, child, atau sibling) dilakukan menggunakan bantuan struktur data Stack saat membaca opening dan closing tag.

Kemudian, teks CSS Selector dari pengguna akan diproses oleh parser.CombinatorParser dan parser.SelectorParser. Modul ini bertugas memecah selector kompleks menjadi aturan-aturan yang bisa dipahami program, seperti Child Combinator (>), Descendant ( ), Adjacent Sibling (+), maupun General Sibling (~), beserta atribut spesifiknya (Class, ID, Tag).

Setelah DOM Tree dan Selector siap, program akan mulai menelusuri pohon dari node root. Sesuai input algorithm, program memanggil traversal.BFS atau traversal.DFS:

- Breadth-First Search (BFS): Program akan menelusuri pohon secara melebar per tingkat (level/depth) menggunakan struktur data queue. Algoritma ini akan mengecek root terlebih dahulu, kemudian semua child dari root, lalu cucu-cucunya, dan seterusnya.
- Depth-First Search (DFS): Program akan menelusuri pohon secara mendalam ke satu cabang hingga paling bawah sebelum berpindah ke cabang saudara menggunakan fungsi rekursif (yang secara implisit bekerja dengan prinsip stack).

Setiap kali mengunjungi sebuah elemen (node), program akan menggunakan modul selector (selector.ComboCombinator) untuk mengevaluasi apakah node tersebut cocok dengan kriteria CSS Selector. Jika cocok, node tersebut akan dimasukkan ke dalam daftar matches (hasil yang dicari). Proses

akan terus berjalan hingga seluruh elemen tertelusuri atau jumlah matches telah mencapai batas maksimum (dari input limit).

Selama proses traversal, setiap elemen yang dikunjungi (visited) dan setiap elemen yang cocok (matched) dicatat dalam sebuah daftar traversal log. Setelah penelusuran selesai:

- Program hitung waktu yang dibutuhkan untuk pencarian (duration), jumlah node yang dikunjungi, jumlah yang cocok, dan mencari kedalaman maksimum (Max Depth).
- Mengubah kembali DOM Tree mentah menjadi struktur representasi JSON (NodeJSON). Node pada JSON ini sudah ditandai mana yang sempat "dikunjungi" (Visited: true) dan mana yang cocok untuk keperluan visualisasi pewarnaan atau animasi di bagian frontend.
- Program juga akan menulis log penelusuran ke dalam sebuah file di folder logs/.
- Semua data ini digabungkan dan dikirim kembali sebagai HTTP Response (format JSON) kepada pengguna.

Berikut pseudocode yang merepresentasikan keseluruhan program :

```
//SCRAPER
FUNCTION Scrape(url):
   req ← BIKIN objek request HTTP GET dengan target url
   SET req.Header "User-Agent" menjadi "Mozilla/5.0..."

   resp ← KIRIM request ke server

   IF resp.StatusCode BUKAN 200 (OK):
      RETURN ERROR

   body ← BACA seluruh isi teks HTML dari resp.Body
   TUTUP koneksi resp.Body

   RETURN body dalam format string

//PARSER
FUNCTION Parse(htmlStr):
   tokenizer ← INISIALISASI pemecah token dari htmlStr
```

```
  virtualRoot ← BUAT Node baru (Tag="#document", Depth=-1)
  stack ← list berisi virtualRoot
  WHILE TRUE:
    token ← AMBIL token berikutnya dari tokenizer
    IF token ADALAH ERROR / End Of File:
      BERHENTI (BREAK LOOP)
    IF token ADALAH StartTag ATAU SelfClosingTag:
      tagName ← BACA nama tag dari token
      parent ← AMBIL elemen terakhir dari stack (TOP)
      node ← BUAT Node baru (Tag=tagName, Parent=parent, Depth=parent.Depth + 1)

      WHILE token memiliki atribut:
        SIMPAN atribut (termasuk id dan class) ke dalam node
      TAMBAHKAN node ke daftar parent.Children
      IF token BUKAN SelfClosingTag:
        TAMBAHKAN node ke belakang stack (PUSH)

    ELSE IF token ADALAH EndTag:
      tagName ← BACA nama tag dari token
      WHILE panjang(stack) > 1:
        top ← AMBIL elemen terakhir dari stack
        HAPUS elemen terakhir dari stack (POP)
        IF top.Tag = tagName:
          BERHENTI (BREAK INNER LOOP)
  root ← CARI child dengan Tag="html" di dalam virtualRoot.Children
  RETURN root

//HANDLER
FUNCTION TraverseHandler(request):
  BACA data JSON dari request (url, html, algorithm, selector, limit)
  IF url TIDAK KOSONG:
    htmlStr ← PANGGIL Scrape(url)
```

```
ELSE IF html TIDAK KOSONG:
   htmlStr ← html
ELSE:
   KEMBALIKAN ERROR "url/html wajib diisi"

root ← PANGGIL parser.Parse(htmlStr)
tokens ← PANGGIL parser.CombinatorParser(selector)

start_time ← CATAT waktu saat ini

IF algorithm = "bfs":
   matches, visited, log ← PANGGIL traversal.BFS(limit, root, tokens)
ELSE IF algorithm = "dfs":
   matches, visited, log ← PANGGIL traversal.DFS(limit, root, tokens)

duration ← HITUNG selisih waktu saat ini dengan start_time

treeJSON ← UBAH root DOM Tree menjadi format JSON
TANDAI elemen di treeJSON yang terdapat di 'visited' dan 'matches'

logFile ← SIMPAN log ke dalam file teks (.log)

response ← BUAT objek JSON berisi:
   - Tree: treeJSON
   - Matches: data matches
   - TraversalLog: log
   - LogFile: path logFile
   - Stats: { VisitedCount, MatchCount, MaxDepth, DurationMs }

KIRIM response ke pengguna (frontend)
```

Pengujian

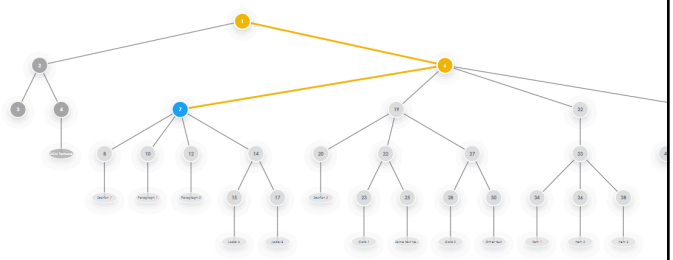
Testcase 1

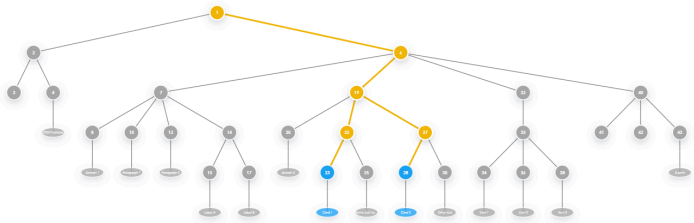
Testcase pertama ini menggunakan input raw HTML, dengan input berikut :

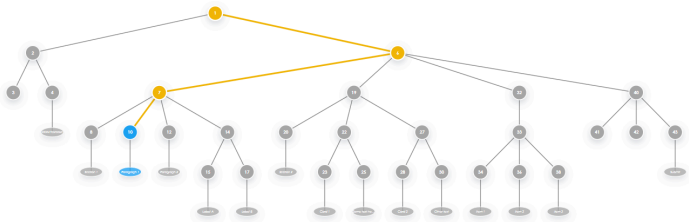
```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>DOM Testcase</title>
</head>
<body>

<section class="section" id="section-1">
  <h2>Section 1</h2>
  <p class="text highlight">Paragraph 1</p>
  <p class="text">Paragraph 2</p>
  <div class="box">
    <span class="label">Label A</span>
    <span class="label special">Label B</span>
  </div>
</section>

<section class="section" id="section-2">
  <h2>Section 2</h2>
  <article class="card" data-type="primary">
    <h3 class="card-title">Card 1</h3>
    <p class="card-text">Some text here</p>
  </article>
  <article class="card" data-type="secondary">
    <h3 class="card-title">Card 2</h3>
    <p class="card-text">Other text</p>
  </article>
</section>

<div class="list-container">
  <ul id="list">
    <li class="item">Item 1</li>
    <li class="item special">Item 2</li>
    <li class="item">Item 3</li>
  </ul>
</div>

<form id="form">
  <input type="text" name="username" placeholder="Username">
  <input type="password" name="password">
  <button type="submit">Submit</button>
</form>
```

```
</body>
</html>
```

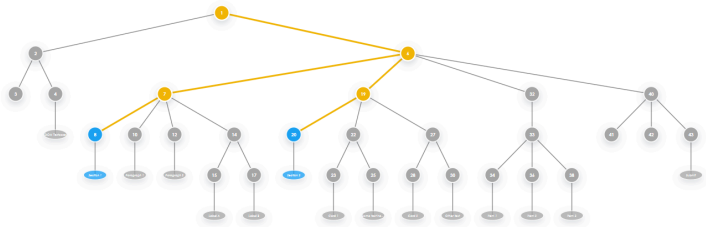
Alg	Input selector	Top-N	Hasil

BFS	section	1	Hasil tree :
			 
			Hasil log :
			`Algorithm : BFS`
			`Selector  : section`
			`Timestamp : 2026-04-25T22:19:00+07:00`
			`Duration  : 0.593 ms`
			`Visited  : 6 nodes`
			`Matched  : 1 nodes`
			`Max Depth : 5`
			`----------`
			`Visit: <html> (depth=0)`
			`Visit: <head> (depth=1)`
			`Visit: <body> (depth=1)`
			`Visit: <meta> (depth=2)`
			`Visit: <title> (depth=2)`
			`Visit: <section#section-1.section> (depth=2)`
			`Match: <section#section-1.section> (depth=2)`

	.card-title	All	Hasil tree :  Hasil log : `Algorithm : BFS`

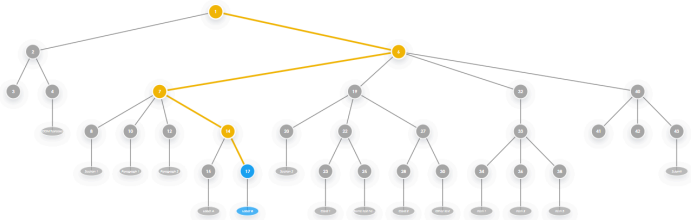
			Selector : .card-title Timestamp : 2026-04-25T22:23:35+07:00 Duration : 0.000 ms Visited : 29 nodes Matched : 2 nodes Max Depth : 5 ---------- - Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <body> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <section#section-1.section> (depth=2) Visit: <section#section-2.section> (depth=2) Visit: <div.list-container> (depth=2) Visit: <form#form> (depth=2) Visit: <h2> (depth=3) Visit: <p.text.highlight> (depth=3) Visit: <p.text> (depth=3) Visit: <div.box> (depth=3) Visit: <h2> (depth=3) Visit: <article.card> (depth=3) Visit: <article.card> (depth=3) Visit: <ul#list> (depth=3) Visit: <input> (depth=3) Visit: <input> (depth=3) Visit: <button> (depth=3) Visit: <span.label> (depth=4) Visit: <span.label.special> (depth=4) Visit: <h3.card-title> (depth=4) Match: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <h3.card-title> (depth=4) Match: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <li.item> (depth=4) Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4)
	h2 + p	All	Hasil tree : 

			Hasil log : Algorithm : BFS Selector : h2 + p Timestamp : 2026-04-25T22:26:16+07:00 Duration : 1.837 ms Visited : 29 nodes Matched : 1 nodes Max Depth : 5 ---------- - Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <body> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <section#section-1.section> (depth=2) Visit: <section#section-2.section> (depth=2) Visit: <div.list-container> (depth=2) Visit: <form#form> (depth=2) Visit: <h2> (depth=3) Visit: <p.text.highlight> (depth=3) Match: <p.text.highlight> (depth=3) Visit: <p.text> (depth=3) Visit: <div.box> (depth=3) Visit: <h2> (depth=3) Visit: <article.card> (depth=3) Visit: <article.card> (depth=3) Visit: <ul#list> (depth=3) Visit: <input> (depth=3) Visit: <input> (depth=3) Visit: <button> (depth=3) Visit: <span.label> (depth=4) Visit: <span.label.special> (depth=4) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <li.item> (depth=4) Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4)
	.section > h2	All	Hasil tree :

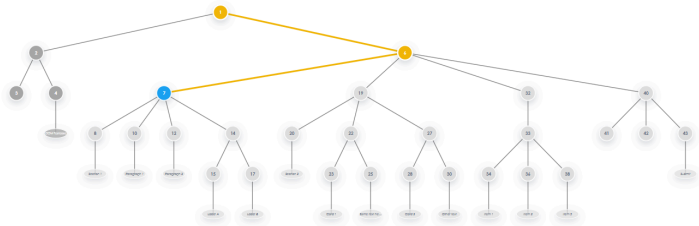
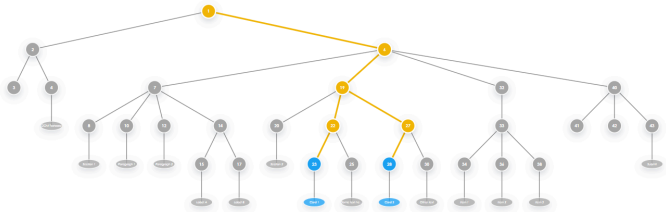


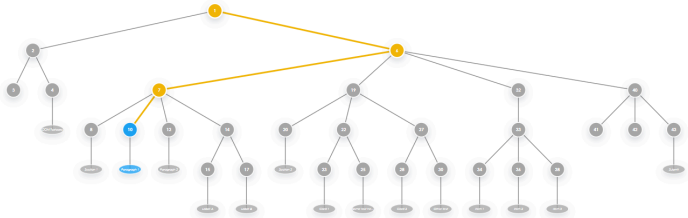
Hasil log :

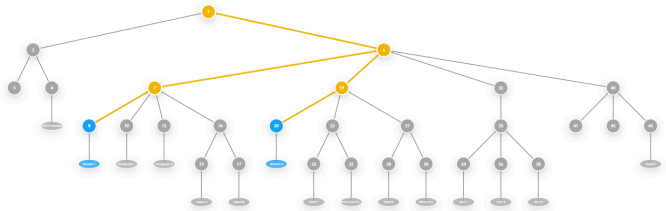
```
Algorithm : BFS
Selector  : .section > h2
Timestamp : 2026-04-25T22:28:52+07:00
Duration  : 0.000 ms
Visited   : 29 nodes
Matched   : 2 nodes
Max Depth : 5
----------------------------------------
-
Visit: <html> (depth=0)
Visit: <head> (depth=1)
Visit: <body> (depth=1)
Visit: <meta> (depth=2)
Visit: <title> (depth=2)
Visit:      <section#section-1.section>
(depth=2)
Visit:      <section#section-2.section>
(depth=2)
Visit: <div.list-container> (depth=2)
Visit: <form#form> (depth=2)
Visit: <h2> (depth=3)
Match: <h2> (depth=3)
Visit: <p.text.highlight> (depth=3)
Visit: <p.text> (depth=3)
Visit: <div.box> (depth=3)
Visit: <h2> (depth=3)
Match: <h2> (depth=3)
Visit: <article.card> (depth=3)
Visit: <article.card> (depth=3)
Visit: <ul#list> (depth=3)
Visit: <input> (depth=3)
Visit: <input> (depth=3)
Visit: <button> (depth=3)
Visit: <span.label> (depth=4)
Visit: <span.label.special> (depth=4)
Visit: <h3.card-title> (depth=4)
Visit: <p.card-text> (depth=4)
Visit: <h3.card-title> (depth=4)
Visit: <p.card-text> (depth=4)
Visit: <li.item> (depth=4)
Visit: <li.item.special> (depth=4)
```

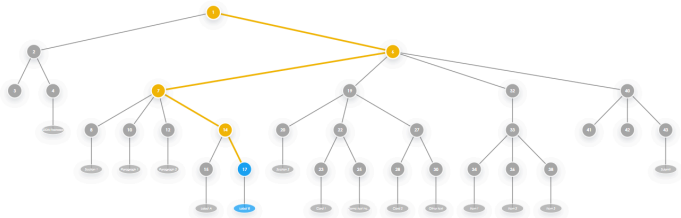
<table>
<tr><td></td><td></td><td></td><td>Visit: <li.item> (depth=4)</td></tr>
<tr><td></td><td>.label ~ .label</td><td>All</td><td>Hasil tree :
 
Hasil log :
Algorithm : BFS
Selector : .label ~ .label
Timestamp : 2026-04-25T22:31:41+07:00
Duration : 0.000 ms
Visited : 29 nodes
Matched : 1 nodes
Max Depth : 5

-
Visit: <html> (depth=0)
Visit: <head> (depth=1)
Visit: <body> (depth=1)
Visit: <meta> (depth=2)
Visit: <title> (depth=2)
Visit: <section#section-1.section> (depth=2)
Visit: <section#section-2.section> (depth=2)
Visit: <div.list-container> (depth=2)
Visit: <form#form> (depth=2)
Visit: <h2> (depth=3)
Visit: <p.text.highlight> (depth=3)
Visit: <p.text> (depth=3)
Visit: <div.box> (depth=3)
Visit: <h2> (depth=3)
Visit: <article.card> (depth=3)
Visit: <article.card> (depth=3)
Visit: <ul#list> (depth=3)
Visit: <input> (depth=3)
Visit: <input> (depth=3)
Visit: <button> (depth=3)
Visit: <span.label> (depth=4)
Visit: <span.label.special> (depth=4)
Match: <span.label.special> (depth=4)
Visit: <h3.card-title> (depth=4)
Visit: <p.card-text> (depth=4)
Visit: <h3.card-title> (depth=4)
Visit: <p.card-text> (depth=4)
Visit: <li.item> (depth=4)</td></tr>
</table>

			Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4)
DFS	section	1	Hasil tree :  Hasil log : Algorithm : DFS Selector : section Timestamp : 2026-04-25T22:36:32+07:00 Duration : 0.000 ms Visited : 6 nodes Matched : 1 nodes Max Depth : 5 ---------- Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <body> (depth=1) Visit: <section#section-1.section> (depth=2) Match: <section#section-1.section> (depth=2)
	.card-title	All	Hasil tree :  Hasil log : Algorithm : DFS Selector : .card-title Timestamp : 2026-04-25T22:37:33+07:00 Duration : 0.000 ms Visited : 29 nodes Matched : 2 nodes Max Depth : 5

			---------- - Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <body> (depth=1) Visit: <section#section-1.section> (depth=2) Visit: <h2> (depth=3) Visit: <p.text.highlight> (depth=3) Visit: <p.text> (depth=3) Visit: <div.box> (depth=3) Visit: <span.label> (depth=4) Visit: <span.label.special> (depth=4) Visit: <section#section-2.section> (depth=2) Visit: <h2> (depth=3) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Match: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Match: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <div.list-container> (depth=2) Visit: <ul#list> (depth=3) Visit: <li.item> (depth=4) Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4) Visit: <form#form> (depth=2) Visit: <input> (depth=3) Visit: <input> (depth=3) Visit: <button> (depth=3)
	h2 + p	All	Hasil tree :  Hasil log : Algorithm : DFS Selector : h2 + p Timestamp : 2026-04-25T22:38:18+07:00 Duration : 0.000 ms Visited : 29 nodes

			Matched : 1 nodes Max Depth : 5 ---------- - Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <body> (depth=1) Visit: <section#section-1.section> (depth=2) Visit: <h2> (depth=3) Visit: <p.text.highlight> (depth=3) Match: <p.text.highlight> (depth=3) Visit: <p.text> (depth=3) Visit: <div.box> (depth=3) Visit: <span.label> (depth=4) Visit: <span.label.special> (depth=4) Visit: <section#section-2.section> (depth=2) Visit: <h2> (depth=3) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <div.list-container> (depth=2) Visit: <ul#list> (depth=3) Visit: <li.item> (depth=4) Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4) Visit: <form#form> (depth=2) Visit: <input> (depth=3) Visit: <input> (depth=3) Visit: <button> (depth=3)
	.section > h2	All	Hasil tree :  Hasil log : Algorithm : DFS Selector : .section > h2 Timestamp : 2026-04-25T22:39:37+07:00

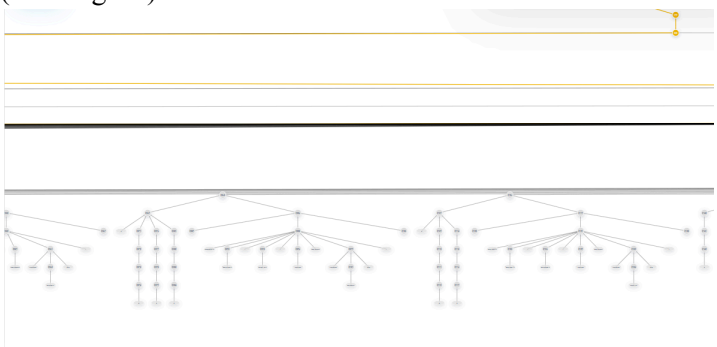
			Duration : 0.000 ms Visited : 29 nodes Matched : 2 nodes Max Depth : 5 ---------- - Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <body> (depth=1) Visit: <section#section-1.section> (depth=2) Visit: <h2> (depth=3) Match: <h2> (depth=3) Visit: <p.text.highlight> (depth=3) Visit: <p.text> (depth=3) Visit: <div.box> (depth=3) Visit: <span.label> (depth=4) Visit: <span.label.special> (depth=4) Visit: <section#section-2.section> (depth=2) Visit: <h2> (depth=3) Match: <h2> (depth=3) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <div.list-container> (depth=2) Visit: <ul#list> (depth=3) Visit: <li.item> (depth=4) Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4) Visit: <form#form> (depth=2) Visit: <input> (depth=3) Visit: <input> (depth=3) Visit: <button> (depth=3)
	.label ~ .label	All	Hasil tree :  Hasil log : Algorithm : DFS

			Selector : .label ~ .label Timestamp : 2026-04-25T22:40:38+07:00 Duration : 0.000 ms Visited : 29 nodes Matched : 1 nodes Max Depth : 5 ---------- - Visit: <html> (depth=0) Visit: <head> (depth=1) Visit: <meta> (depth=2) Visit: <title> (depth=2) Visit: <body> (depth=1) Visit: <section#section-1.section> (depth=2) Visit: <h2> (depth=3) Visit: <p.text.highlight> (depth=3) Visit: <p.text> (depth=3) Visit: <div.box> (depth=3) Visit: <span.label> (depth=4) Visit: <span.label.special> (depth=4) Match: <span.label.special> (depth=4) Visit: <section#section-2.section> (depth=2) Visit: <h2> (depth=3) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <article.card> (depth=3) Visit: <h3.card-title> (depth=4) Visit: <p.card-text> (depth=4) Visit: <div.list-container> (depth=2) Visit: <ul#list> (depth=3) Visit: <li.item> (depth=4) Visit: <li.item.special> (depth=4) Visit: <li.item> (depth=4) Visit: <form#form> (depth=2) Visit: <input> (depth=3) Visit: <input> (depth=3) Visit: <button> (depth=3)

Testcase 2

Testcase ke-2 ini menggunakan input Web dengan sumber :
https://en.wikipedia.org/wiki/6-7_meme

Alg	Input selector	Top-N	Hasil

BFS	p > a	5	Hasil tree : (terlalu gede)  Hasil log : hasil input web
DFS	p > a	5	Hasil tree : Terlalu besar Hasil log : hasil input web

Bab V: Kesimpulan dan Saran

Kesimpulan

Aplikasi Sbrep2 DOM Traversal berhasil dibuat untuk melakukan web scraping menggunakan algoritma BFS dan DFS. Aplikasi dapat menerima input berupa Website URL atau Raw HTML, lalu mencari elemen berdasarkan input CSS selector.

Algoritma BFS dan DFS berhasil diterapkan dalam proses traversal DOM. BFS menelusuri node secara melebar, sedangkan DFS menelusuri node secara mendalam. Hasil pencarian ditampilkan dalam bentuk data hasil scraping, traversal log, waktu pencarian, jumlah node yang dikunjungi, dan visualisasi pohon, serta visualisasi animasi pencarian.

Dengan demikian, aplikasi telah memenuhi tujuan utama tugas besar, yaitu menerapkan algoritma BFS dan DFS pada kasus web scraping.

Saran

Aplikasi masih dapat dikembangkan dengan menambahkan dukungan untuk website dinamis yang menggunakan JavaScript. Selain itu, hasil scraping dapat dibuat agar bisa diekspor ke format seperti CSV atau JSON.

Lampiran

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi		✓
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting		✓

Tabel Pembagian Tugas

NIM	Nama	Tugas
13524022	Ishak Palentino Napitupulu	- Frontend
13524112	Richard Samuel Simanullang	- Traversal (BFS dan DFS) - CSS Selectors
13524118	Dylan Gregory Tondang	- Traversal (BFS dan DFS) - CSS Selectors

Link Video: https://youtu.be/HSTVF65Xt2E
Kode Sumber:

FrontEnd : https://github.com/Lloyd565/Tubes2_sbrep2_FE
BackEnd : https://github.com/Lloyd565/Tubes2_sbrep2_BE

Daftar Pustaka

Rinaldi. 2026. *Algoritma BFS dan DFS (Bagian 1)*. Informatika. Diperoleh dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2025)-Bagian1.pdf pada 20 April 2026 Pukul 13.00 WIB

https://ramdannur.wordpress.com/2020/11/10/data-structure-mengenal-graph-tree/

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Ishak Palentino Napitupulu

Richard Samuel Simanullang

Dylan Gregory Tondang